Design Add and Search Words Data Structure (medium)

Problem Statement

Design a data structure that supports the addition of new words and the ability to check if a string matches any previously added word.

Implement the `Solution` class:

- `Solution()` Initializes the object.
- `void addWord(word)` Inserts `word` into the data structure, making it available for future searches.
- `bool search(word)` Checks if there is any word in the data structure that matches `word`. The method returns `true` if such a match exists, otherwise returns `false`.

Note: In the search query `word`, the character `'.'` can represent any single letter, effectively serving as a wildcard character.

Examples

Example 1:

- **Input**:

```
["Solution", "addWord", "addWord", "search", "search"]
[[], ["apple"], ["banana"], ["apple"], ["....."]]
```

- **Expected Output**:

```
[-1, -1, -1, 1, 1]
```

- Justification: After adding the words "apple" and "banana", searching for "apple" will return `true` since "apple" is in the data structure. Searching for "....." will also return `true` as both "apple" and "banana" match the pattern.

Example 2:

- **Input**:

```
["Solution", "addWord", "addWord", "search", "search"]
[[], ["cat"], ["dog"], ["c.t"], ["d..g"]]
```

- **Expected Output**:

```
[-1, -1, -1, 1, 0]
```

- **Justification**: "c.t" matches "cat" and "d..g" doesn't matches "dog".

Example 3:

- **Input**:

```
["Solution", "addWord", "search", "search"]
[[], ["hello"], ["h.llo"], ["h...o"]]
```

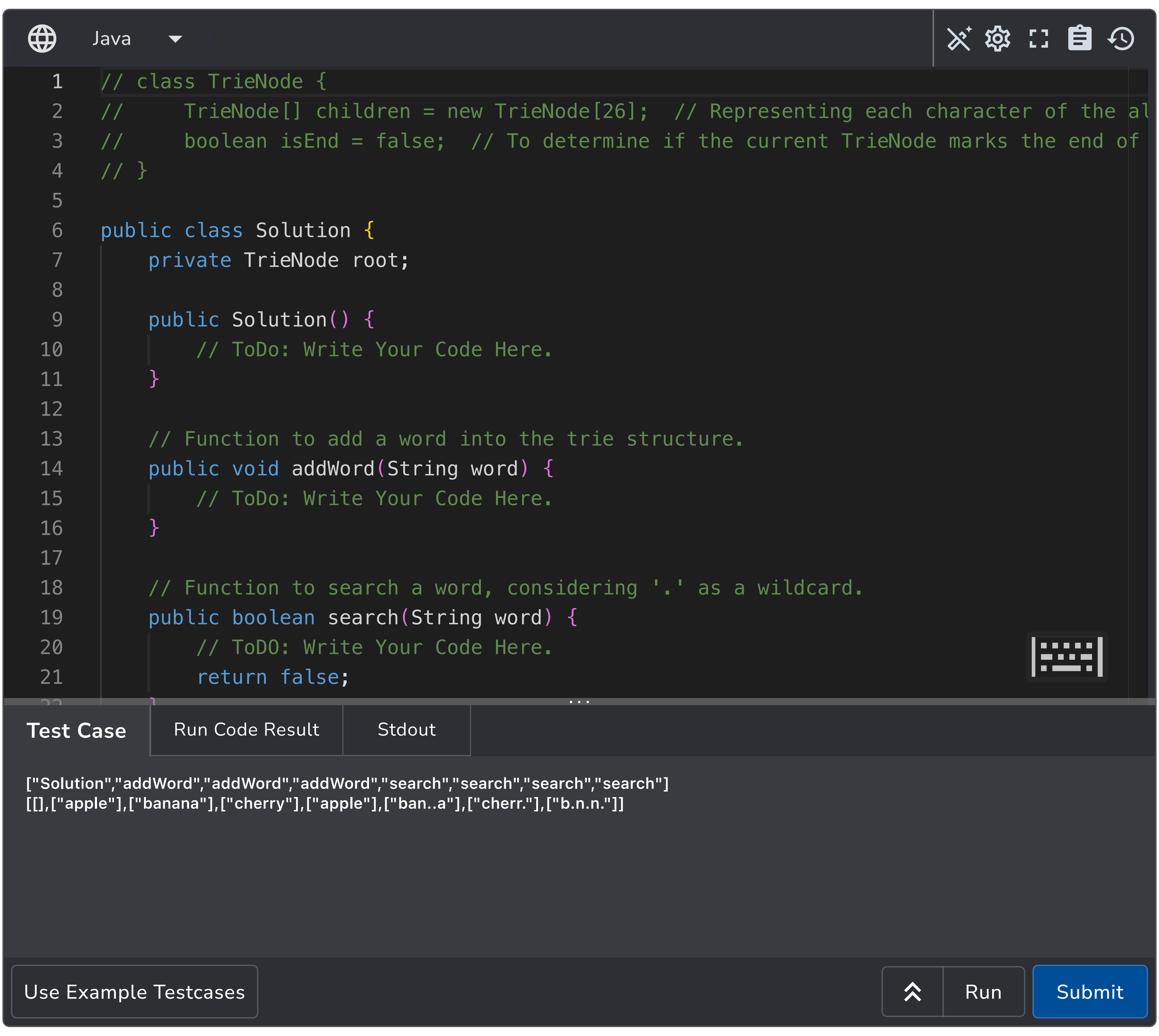
- **Expected Output**:

```
[-1, -1, 1, 0]
```

- **Justification**: "h.llo" and "h...o" both match "hello".

Try it yourself

Try solving this question here:



```java
// class TrieNode {
//     TrieNode[] children = new TrieNode[26];  // Representing each character of the al
//     boolean isEnd = false;  // To determine if the current TrieNode marks the end of
// }

public class Solution {
    private TrieNode root;

    public Solution() {
        // ToDo: Write Your Code Here.
    }

    // Function to add a word into the trie structure.
    public void addWord(String word) {
        // ToDo: Write Your Code Here.
    }

    // Function to search a word, considering '.' as a wildcard.
    public boolean search(String word) {
        // ToDO: Write Your Code Here.
        return false;
```

```
["Solution","addWord","addWord","addWord","search","search","search","search"]
[[],["apple"],["banana"],["cherry"],["apple"],["ban..a"],["cherr."],["b.n.n."]]
```

← Previous
Solution: Index Pairs Of A String

Next →
Solution: Design Add And Search Words Data Structure

☐ Mark as Completed